

UNIVERSITY OF WATERLOO

FACULTY OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING

ECE 498A

Automatic Air Hockey Robot Detailed Design Document

(names removed)

One of the notable strengths of this report is the quantitative technical analysis in Section 3.1.

Table of Contents

1	High Level Description of Project	1
1.1	Motivation	1
1.2	Project Statement	1
1.3	Block Diagram	2
2	Project Specifications	3
2.1	Functional Specifications	3
2.2	Non-Functional Specifications	4
3	Detailed Design	5
3.1	Linear Actuator Design	5
3.1.1	Final Design	5
3.1.2	Engineering Design Process	8
3.2	Camera Mount Design	9
3.2.1	Final Design	9
3.2.2	Engineering Design Process	10
3.3	Paddle Control Unit Design	11
3.3.1	Final Design	11
3.3.2	Engineering Design Process	13
3.4	Robotic Controls Design	13
3.4.1	Final Design	13
3.4.2	Engineering Design Process	14
3.5	Tracking Algorithm Design	16
3.5.1	Final Design	16
3.5.2	Engineering Design Process	18
3.6	Game State Algorithm Design	20
3.6.1	Final Design	20
3.6.2	Engineering Design Process	21
4	Discussion and Project Timeline	24
4.1	Discussion on Design	24
4.1.1	Hours Worked	24
4.1.2	Critical Design Evaluation	24
4.1.3	Potential Hazards	25
4.2	Project Timeline	26
	References	28

1 High Level Description of Project

1.1 Motivation

Finding someone to play a game of air hockey is not always easy, and challenging opponents can be difficult to come by. The Automatic Air Hockey Robot is designed to hone the strategy, strength and mental abilities of players [1] when a human opponent is unavailable.

The field of robotic vision processing is evolving and progressing rapidly. In order to create a viable and competitive robotic air hockey application, extensive use of optimized video processing is needed. This project would then serve as an example implementation of applied, high precision robotic vision and control using inexpensive, readily available components. The specific application of an air hockey table is not novel, but existing implementations either use expensive parts [2] or interface with a playing surface too small for regular play [3].

1.2 Project Statement

The objective of this project is to design a system that automatically plays air hockey against an opponent, simulating a skilled player. This involves having an accurate puck detection system, a system that controls the paddle and efficient algorithms that are used to ensure that the paddle is moved into locations that can intercept the puck in a reasonable time frame.

1.3 Block Diagram

Figure 1 displays the planned block diagram system for the Automatic Air Hockey Robot. The system is designed to use feedback in a closed control loop, where the position of the paddle and puck determine future movements to be made by the robotic system. The processing unit takes individual frames from the camera as input, determines the position of the puck on the table, and changes the location of the paddle based on this information.

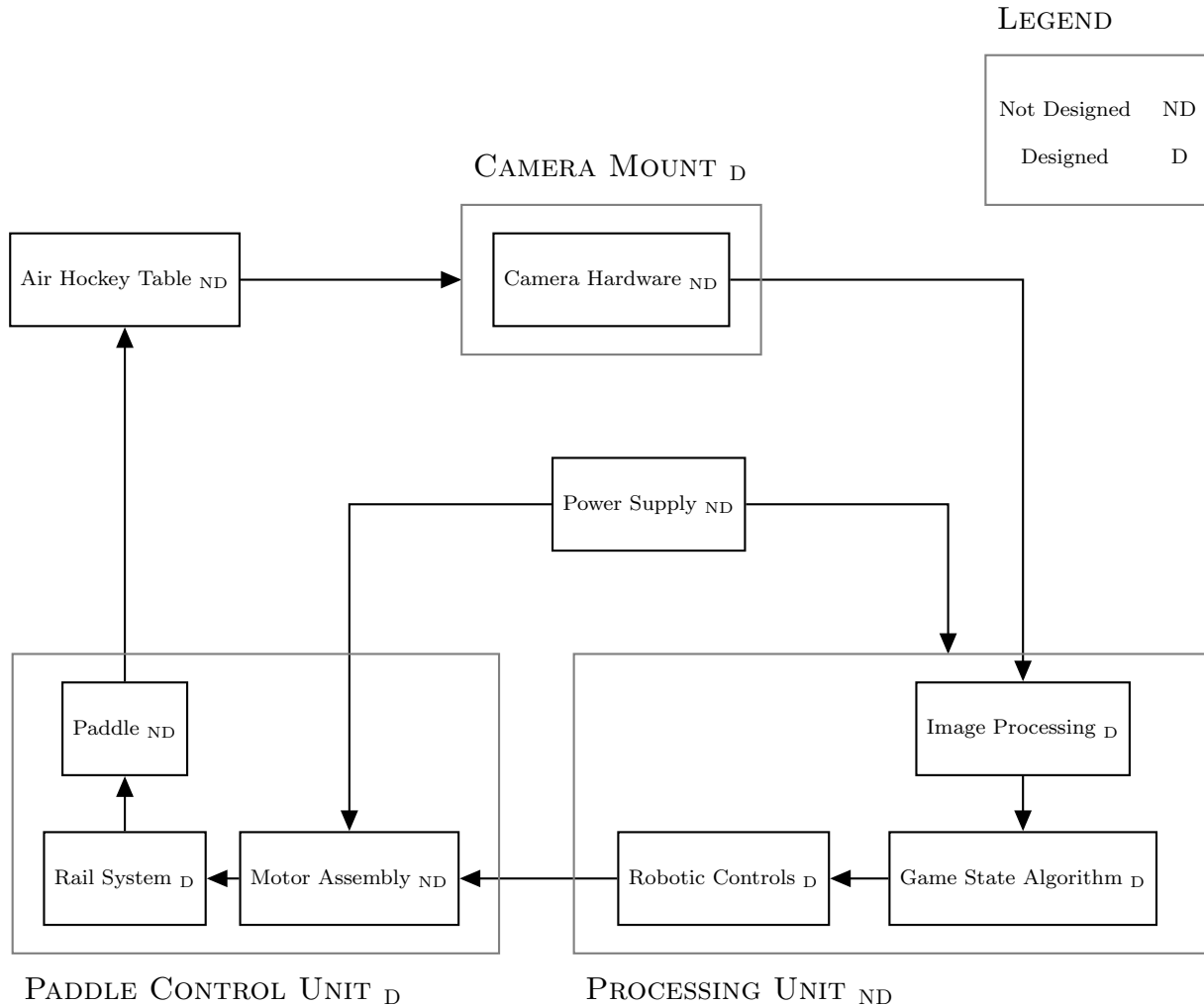


Figure 1: Block Diagram of the Complete System

2 Project Specifications

2.1 Functional Specifications

Table 1 displays the non-functional requirements determined to be needed for the development of the Automatic Air Hockey Robot.

Table 1: Functional Specifications

Subsystem	Specification	Essential
Camera Mount	Must not inhibit play area	Y
Camera Mount	Mount structure must be able to support weight of itself and the camera	Y
Camera Hardware	Must be able to distinguish between different colors used for identification of table components (e.g. puck, sides)	Y
Processing Unit	Must accept input in given format from camera	Y
Processing Unit	Must interface with paddle control unit	Y
Processing Unit	Must successfully receive all video data sent from camera	Y
Image Processing Algorithm	Must be able to identify distinct colors used for identification of table components	Y
Image Processing Algorithm	Must predict puck position within $\pm 25\%$ of diameter of puck within $\frac{1}{60}$ s	Y
Game State Algorithm	Must keep state of puck position, updating at a rate of at least 60 Hz	Y
Game State Algorithm	Must be able to suggest position of paddle to intercept puck every $\frac{1}{60}$ s based on current position	Y
Game State Algorithm	Must support a defensive play mode in which the paddle moves along the goal line in one dimension to block shots	Y
Game State Algorithm	Must support an aggressive play mode in which the paddle can move in two dimensions to intercept the puck with a trajectory towards the opposing goal	N
Motor Assembly	Must be able to move paddle at velocities up to 5 m/s	Y
Motor Assembly	Must be able to position paddle with steady state error of $\pm 25\%$ of puck diameter	Y

Table continues on next page.

Subsystem	Specification	Essential
Rail System	Must keep position of paddle steady when struck by puck at 5 m/s	Y
Rail System	Must support motion of paddle in two dimensions	N
Power System	Must provide adequate power to support functionality of all other systems requiring project	Y
Robotic Control	Must interface with motor, providing output signals within acceptable voltage range	Y
Robotic Control	Must transform position reference value into motor input signal	Y

2.2 Non-Functional Specifications

Table 2 displays the functional specifications determined for the development of the Automatic Air Hockey Robot.

Table 2: Non-Functional Specifications

Subsystem	Specification	Essential
Air Hockey Table	System must operate on a commercially available air hockey table with minimum play area dimensions of 5 ft x 2 ft	Y
Camera Mount	Must be able to view half of the play area of the table, at minimum, a 2.5 ft x 2 ft area	Y
Camera Hardware	Must have a high enough frame rate to identify puck moving at 7.5 m/s	Y
Camera Hardware	Must have a resolution large enough to correctly identify puck when fixed in mount system	Y
Processing Unit	Must have a clock cycle fast enough to prevent delays face with programs running on it	Y
Rail System	Must be mountable on table	Y
Rail System	Must support motion over play area of board, minimum 2 ft x 2 ft	Y

3 Detailed Design

3.1 Linear Actuator Design

A linear actuator is used to create motion in a straight line and so is being used to move the paddle across the table.

3.1.1 Final Design

There were a number of different types of linear actuators considered for the design. However, the final design uses an electro-mechanical actuator which uses a stepper motor as the control method for the rail. The advantage of using a stepper motor is that it allows for precise control of position.

The stepper motor that drives the actuator is a 12 V motor manufactured by Soyo [4]. The motor has a resolution of 1.8° per step and a holding torque of 0.88 N m. Table 3 shows the parts list for the actuator; all parts were sourced from the Open Builds Part Store [5].

Table 3: Parts List for Linear Actuator

Subsystem	Part	Quantity
Belt System	Smooth Idler Pulley Kit	1
Belt System	GT2 (2 mm) Timing Belt	6
Belt System	GT2 (2 mm) Aluminum Timing Pulley - 30	1
Rail	V-Slot 20 x 40 mm (Length: 750 mm)	1
Plate	OpenRail Gantry Plate	1
Miscellaneous Hardware	Socket Head Screws M3 (25 Pack) (Length: 6 mm)	1
Miscellaneous Hardware	Aluminum Spacers	4
Miscellaneous Hardware	Eccentric Spacers	2
Miscellaneous Hardware	Low Profile Screws M5 (25 Pack) (Length: 6 mm)	1
Miscellaneous Hardware	Low Profile Screws M5 (10 Pack) (Length: 30 mm)	1
Miscellaneous Hardware	Mini V Precision Shim - 8 x 5 x 1 mm	4
Miscellaneous Hardware	Belt Crimp Clamp	2
Miscellaneous Hardware	M3 Cap Head Screws (Length: 30 mm)	4
Miscellaneous Hardware	End Cap (V-Slot)	2
Miscellaneous Hardware	V-Slot Actuator End Mount	2

The parts listed above can be assembled to build the linear actuator. Figure 2 is a render of what the final product will look like, and 3 is a CAD representation of the system with dimensions. Some portions of the design were taken from files available at [6].

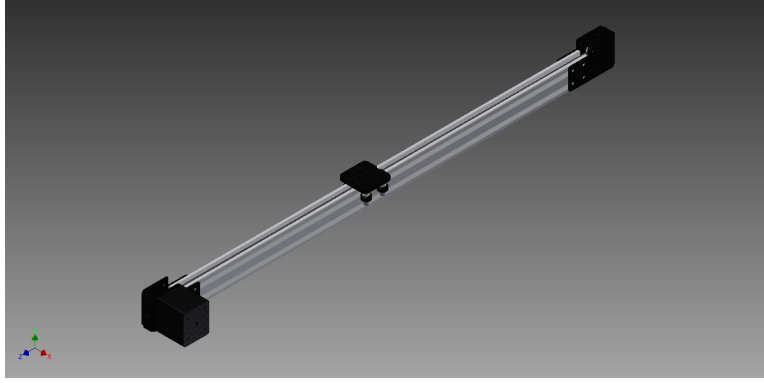


Figure 2: Linear Actuator Mock-Up

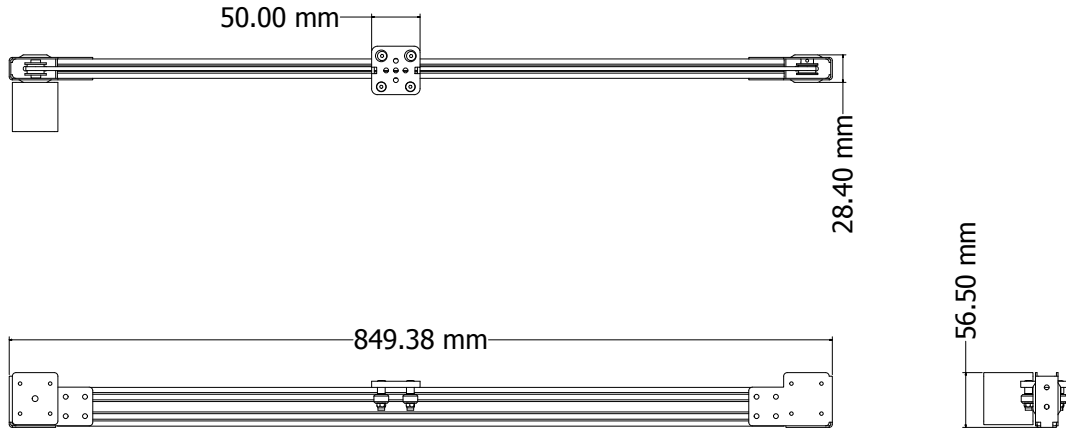


Figure 3: Linear Actuator CAD Drawing

The following calculations were carried out to ensure that the selected motor met the necessary specifications. The first necessary calculation was the load torque T_L , expressed as follows:

$$T_L = \frac{FD}{2\eta i}$$

Where F = force of moving direction, D = final pulley diameter (m), η = efficiency and i = gear ratio. The Soyo motor has an efficiency of 95%, the gear assembly used has a gear ratio of 1, and the diameter of the pulley is 15 mm. Using these values in the formula above, the calculated load torque is:

$$T_L = \frac{0.5 * 0.015}{2 * 0.95 * 1} = 0.004 \text{ N m}$$

The next calculation to make is the acceleration torque. This is necessary due to the fact that the motor has a varying speed. The acceleration torque can be calculated as follows, where J_0 is the rotor inertia, J_L is the total load inertia, θ_S is the step angle, $n = \frac{3.6}{(\theta_S * i)}$ and f is the operating pulse speed:

$$\begin{aligned} T_A &= (J_0 i^2 + J_L) * \frac{\pi \theta_S}{180 n} * f^2 \\ &= (0.00002 + 0.5) * \frac{\pi * 1.8}{180 * 2} * 8^2 \\ &= 0.503 \text{ N m} \end{aligned}$$

Finally, the total motor torque required is the acceleration torque added to the load torque to get

$$\begin{aligned} T_M &= 0.503 + 0.004 \\ &= 0.507 \text{ N m} \end{aligned}$$

Therefore, a minimum torque of 0.507 N m is required to be able to move the paddle. The selected motor has a torque of 0.88 N m, and therefore is a good fit for the system. Having additional torque as a safety factor is also important to allow for flexibility in the design. In this situation, the motor torque is about 1.5 times greater than the minimum amount necessary, and so should be able to handle additional strain to the system.

To ensure that the motor is driven accurately, the Big Easy Driver, manufactured by SparkFun [7], will be used. The driver circuit can handle the Soyo motor's rated specifications without any issues, as it can supply a current of up to 1.4 A per phase. The Soyo motor used in the design only requires 0.68 A. The driver circuit also contains on-board voltage regulation, and so only requires one supply voltage to power both the electronics and the motor. It also has the ability to interface with Arduino microcontrollers, which will be discussed in more detail in the Robotic Controls Design section.

The design of the actuator allows for precise movements of the belt, falling within the project specifications listed for paddle movement resolution. The following equation calculates the linear distance traveled by the belt system when the motor turns one step, where $p = 2 \text{ mm}$ is the timing pitch, $t = 36$ is the number of teeth on the timing pulley, and $p_d = pt$ is the pitch diameter.

$$\begin{aligned}\text{step size} &= \frac{p_d}{\frac{360^\circ}{d}} \\ &= \frac{72}{200} = 0.36 \text{ mm}\end{aligned}$$

And the linear distance traveled per step is

$$\begin{aligned}&\frac{1 \text{ mm}}{\text{step size}} \\ &= 2.788 \text{ mm/step}\end{aligned}$$

This motor and actuator system should meet the requirements that were laid out in the specifications, and should also be able to drive the paddle without any issues.

3.1.2 Engineering Design Process

The motor selection process involved a number of steps to reach the final design solution. The first step was researching the specific type of motor that was going to be used. Two options for the type of motor used were servo and stepper motors. Servo motors have the advantage of providing feedback to the controller with regards to its exact position. Servo motors have the advantage of having a method of providing feedback to the controller with regards to its exact position. Furthermore, servo motors are generally considered to have higher performance with regards to speed. However, they generally also require slightly more complicated setup since they require pulse width modulation (PWM) tuning.

Stepper motors, on the other hand, are much better for incremental position adjustments. Servo motors generally require a feedback mechanism to drive positioning, however stepper motors have inherent position control due to its nature of operation. The motors rotate in fixed fractional increments, and so are much more precise in moving a specific distance. Stepper motors also provide a constant holding torque without the need for the motor to be powered. Therefore, once a motor has been moved into place, it does not require additional power to ensure it holds that position. Due to the nature of the project, where precise positioning is required to intercept the puck, a stepper motor was better suited to meet the design criteria.

Initial testing involved using a simple stepper motor and driver in conjunction with an Arduino microprocessor to understand how stepper motors can be controlled and how quickly they can be driven. Testing was carried out on a number of different motors with different voltage, current and torque ratings to understand which performance characteristics would be important in the design. It was clear that the tested motors did not have enough torque to move the paddle at the required speeds. All of the tested motors had torque well below 0.5 N m, and so calculations were carried out to determine what the minimum required torque would be. The motors also had very low current ratings, generally limited to less than 0.5 A per phase, and so would not be able to accelerate quick enough. From these tests, it was

determined that the Soyo motors would be used in the final design.

The design for the linear actuator was fairly straightforward. Electro-mechanical actuators are very common and generally have a fixed design. The major decisions made were related to sizing the rail and belt system correctly. The initial design was based on a system that used threaded rods instead of a timing belt/pulley solution. Threaded rods are generally slower, but more precise than timing belts. Since the actuator had to move the paddle over a relatively large area and be able to keep up with the speed of the puck, it was decided that a belt system would be used. An alternative to the belt system that maintained a high speed was using ACME rods, however these systems are generally much more expensive than the other two alternatives, and would not have provided a significant speed advantage over the timing belt system.

3.2 Camera Mount Design

A simple mount is to be constructed to house the camera that is to be used for the object tracking.

3.2.1 Final Design

The camera mount will be a simple wooden frame that is attached to the centre of the air hockey table. This is a cost-effective solution that will also match the original design of the table itself. The main purpose of the mount is to hold the camera that is to be used for the image processing algorithm discussed in Section 3.5.

The choice of wood as the material used to construct the mount system was selected simply because of its ease of use. The mount system, while important, is a relatively simple portion of the overall system, so any time that can be saved in its construction can be spent on more work intensive subsystems. An added benefit of using wood is that the table being used as the basis of the project already has wood veneer around it, so a wooden structure will match the table well. The mount will be attached to the sides of the air hockey table at the centre line. It will be approximately 1 metre high from the top of the table. This positioning and height will prevent the mount system from interfering with the play area in any way and also allow the camera to view the required space on the table. The height was determined by testing the field of view of the camera at various heights. To ensure that the mount is stable and can support itself and the camera, it will have angled braces in the top corners to add extra support between the sides and top of the mount. For an example of what this mount will look like, see Figure 4 below.

The camera being used for the the object tracking is a PlayStation Eye. This camera was chosen for its high frame rate and its affordable cost. The PlayStation Eye can operate with a maximum frame rate of 120 frames per second [8], which is extremely high for the cost of the camera. Through experimentation, it was determined that a frame rate this high would be needed to track the puck moving at its maximum speed. The PlayStation Eye connects to a PC through a standard USB port and can be used in the same manner as any

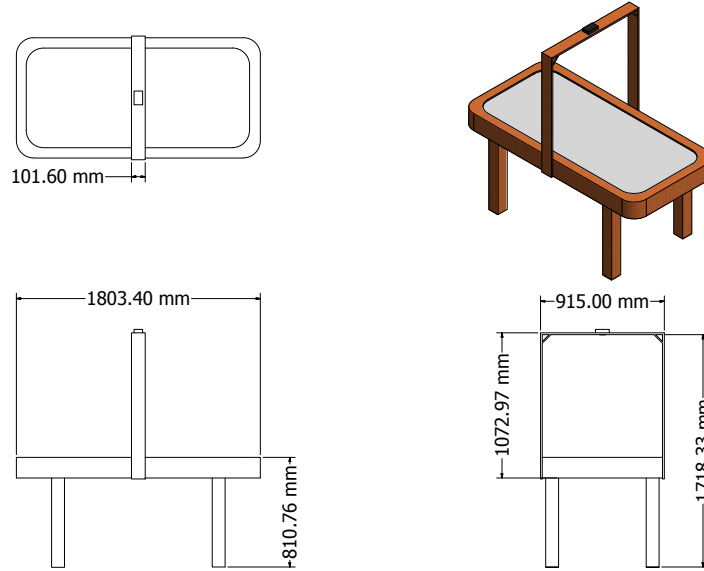


Figure 4: Table With Camera Mount

other webcam. This means that the amount of overhead required to get the camera running was negligible, and also that the required cabling is cheap and readily available. The camera is mounted far from the other processing subsystem hardware, however, as USB extension cables are readily available, any additional resources needed should be easy to find.

The original design also included a lighting system to help ensure a consistent lighting distribution across the table, but after testing the basic object tracking and researching potential lighting options, it was determined that the benefit from such a system would be negligible, and as such, it has been removed from the final design.

3.2.2 Engineering Design Process

The mounting system is a simple design unit, and thus it was relatively simple to determine the final design. Several options were investigated, and certain aspects of the design changed over time.

One of the main changes that occurred during the design process was the removal of the requirement of a lighting unit. When the project specifications were first being outlined, the testing was being done in a very poorly lit environment, and thus it was deemed necessary to have a custom lighting solution to ensure that the tracking software would be able to find the puck. This was changed when additional testing was done in better lighting conditions. The system should be able to function as expected in any moderately lit area without any additional lighting. It was also found that it would be quite difficult to get an adequate lighting solution that would not interfere with the tracking camera. This was because the surface of the table is very glossy and direct lighting causes hot spots on the play area which

would likely be a detriment to the quality of the object tracking and therefore position prediction.

Another important design decision was what type of camera should be used for the object tracking. This decision was made early in the design process so that work on the tracking code could begin at an early stage in the development. Through testing of real world game speeds, it was determined that a high frame rate would be necessary to capture puck movement correctly. However, most high-speed cameras are very expensive, and their benefits do not justify the additional cost. The ease of use of the camera was also an important consideration. If too many specialized tools were required to operate the selected camera at a high frame rate, it would have slowed down the completion of the entire project. Through research online, it was determined that the PlayStation Eye camera could operate at speeds of 120 frames per second, and could be used with a computer in the same manner as any plug-and-play webcam [8]. This suited the needs of the project perfectly, and thus it was chosen to be the camera used in the final design.

The final design decision was for the mount itself. Since the camera being used was a PlayStation Eye, and the lighting system was deemed unnecessary, the weight requirements of the mount system were very low. This meant that the decisions of what materials to use and how to build the structure were mostly based on aesthetics and ease of implementation. The table being used for the project has both black plastic as well as wood veneer along its body, meaning either of those two materials would likely look like they belonged on the table and were not simply put there haphazardly. Out of these two options, wood seemed like a better choice, since it is much easier to work with, and more easily replaceable should it break. The only considerations that remained were ensuring the stability of the structure, and ensuring that the camera could be mounted to it without having the structure obscure the view in any way. How this is done simply depends on the wood used in the structure. Thin boards will be less structurally sound, but allow for the camera to sit on top of the structure with a hole drilled for the lens to fit through. The alternative is that the camera simply be mounted on the underside of the cross bar, which allows for the use of thicker boards to ensure stability. Since the second option will provide a more robust solution, it was chosen.

3.3 Paddle Control Unit Design

3.3.1 Final Design

The complete paddle control unit involves the use of 2 linear actuators as discussed earlier. Each linear actuator operates on a separate axis. Both linear actuators will be combined into an XY table using cylindrical rails. These rails will be mounted to the gantry plate on the actuator, which will then be attached to the air hockey paddle, as described below. The opposite end of each rod will have a wheel that can simply slide along a rail similar to the rails used in the actuator build. This full XY table meets the design requirement of allowing the paddle to be moved in 2 dimensions. Figure 5 is a CAD representation of the two linear actuators combined along with a mounted paddle.

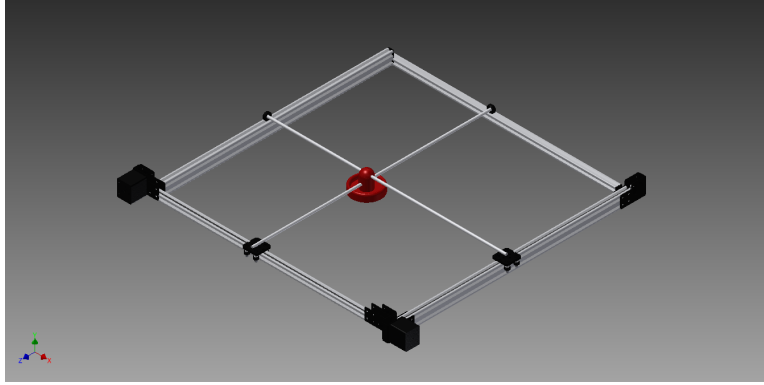


Figure 5: XY Table Mock-Up

To mount the paddle to the rails, the simplest solution involves drilling holes into the paddle and then using linear ball bearings to allow for the paddle to move over cylindrical rails that are mounted to the extrusions of the linear actuator. The XY table would be mounted over the trim of the air hockey table. Figure 6 illustrates the complete system with the XY table mounted.



Figure 6: Full System Mock-Up

The complete system meets all the design requirements, as it enables the paddle to move over the required play dimensions, while not inhibiting gameplay.

3.3.2 Engineering Design Process

The biggest design decision was finding a method of mounting the XY table onto the air hockey table. The two options considered were to mount the XY table along the side of the air hockey table or along the trim around the play area. Using the side of the table would have posed additional challenges, as the extrusion rails of the linear actuator would need to sit over the play area, and so mounting the XY table on the side would require additional mechanical interfacing between the motor and the linear actuator to allow for a correct setup.

The air hockey table has a trim around the side of the play area large enough for the linear actuator to sit on top of. The only challenge involved in this design would be ensuring that the actuators are mounted in a robust manner. However, this was a much easier problem to solve, requiring less mechanical design and so the trim mount was chosen for the final system.

3.4 Robotic Controls Design

This system is used to control the motor assemblies of the paddle control unit using reference paddle position data from the game state algorithm.

3.4.1 Final Design

The final design of the motor utilizes one feedback loop in the full system using the camera. While stepper motors by design can be used in an open loop feedback system, it is prudent to also design the system to withstand drift over time due to missed steps. In a stepper motor, missed steps will occur when the operating range of the current and voltage is either too far below or above the rated values of the stepper motor.

The controls software will incorporate the feedback loop in the following manner. The camera system will track both the puck and paddle system, and will communicate the position of the paddle and the puck to the Arduino controls software. As such, the Arduino controls software will know its position subject to error and drift, and where the camera system sees the paddle unit. Thus, the software will be able to adequately handle the movement to the correct position on the table with the given feedback information.

The design will communicate with the game state algorithm, receiving position data as (x, y) co-ordinates sent via a serial connection as 8 bit characters. This position data references a grid on the table, and with 8-bit data, allows for position precision within 3 mm. The master process will send positional information for movement details and also for feedback control of the paddle unit. The expected data format may look similar to the following:

00 FE 64 65

Where the first underlined section corresponds to (0, 254), and the second (100, 101) on the grid system.

The Arduino will utilize the AccelStepper [9] library, an open source stepper motor library designed for use with the Arduino platform which significantly decreases development time for the motor control software. The AccelStepper library uses ramping [10] based on the acceleration provided to smoothly accelerate a motor to the correct speed.

The controls software will provide a simple application programming interface that exposes a method that allows the programmer to pass the correct co-ordinates as an (x, y) pair. The state and position of the paddle unit is tracked internally and with the feedback systems mentioned above, corrected over time to eliminate drift. In psuedo-code, some example functions for movement and drift correction may look as follows:

```

1      char currentX, currentY;
2
3      void moveTo(char x, char y) {
4          int newX = findStepDifference(x, oldX);
5          int newY = findStepDifference(y, oldY);
6          moveXAxis(newX);
7          moveYAxis(newY);
8      }
9
10     void moveRelative(char x, char y) {
11         moveXAxis(x);
12         moveYAxis(y);
13     }
14
15     void correctDrift(char knownX, char knownY) {
16         char differenceX = knownX - currentX;
17         char differenceY = knownY - currentY;
18         moveRelative(differenceX, differenceY);
19     }

```

3.4.2 Engineering Design Process

Additional Feedback Control

Nothing In this option, the stepper motor works in an open-loop configuration. When running with the rated voltage and amperage, the motor should not skip any steps. This option would require the unit to calibrate on a system restart, as the internal step count is only relative to a known starting position.

Rotary Encoder A rotary encoder can translate the angular motion of the stepper motor to create a digital representation of the position of the motor. Connected to the motor, the continuously turning encoder could be used to create a closed loop feedback system with

the Arduino. This option does not effectively work with the stepper motor however, as a stepper motor keeps track of the relative position by counting the number of expected steps taken. An encoder would allow for the system to operate in a simple feedback loop that would mitigate any steps missed by the motor. However, because the expected load of the system is low at 500 g, running the motor at the rated amperage and voltage will effectively minimize problems related to missed steps and drift over time [11].

Camera Detection Instead of wiring the feedback system directly to the Arduino, the existing camera and image processing algorithm could be used to continuously track the paddle and puck position. Extra information regarding the measured position of the paddle unit can be sent to the robotic controls software to reconcile any position differences if they exist.

Decision While a stepper motor can be used in an open loop system, it cannot know the absolute position relative to the table. Therefore, a closed loop system is needed to allow the system to know the correct position of the paddle unit. While a rotary encoder or multi-turn potentiometer could be connected to the motor, it will only provide relative position information to correct missed steps. Instead, by using the existing camera setup, missed steps can be accounted for, and position on the table can be found reliably.

Data Length The table of play can be split up into an uniform Cartesian grid of a given size. Using the C `int` data type would provide 32 bits of information per axis. In practice, this is much more precise than necessary. Each axis is approximately 75 cm, and using 32 bits of information would result in positions at the nanometer scale. Practically, this is a waste of computational space. Instead, using only 8 bits of information would give a resolution of approximately 2.9 mm which is of the same order as the resolution for the full step of the belt driven axis. This 2.9 mm resolution is well within the bounds set in the specifications of being able to move within $\pm 25\%$ of the diameter which is approximately 24 mm. The calculations below show the resolutions obtained by using 32 bits of information and 8 bits of information to store the positions.

$$\frac{0.75 \text{ m}}{2^{32}} = 0.17 \text{ nm}$$

$$\frac{0.75 \text{ m}}{2^8} = 2.93 \text{ mm}$$

In addition, the usage of micro-stepping up to $\frac{1}{16}$ allows for an additional precision factor of 16, although this will cause the speed and torque of the system to decrease [12]. The incremental torque of a single microstep is as follows, where μ_{pfs} denotes the number of microsteps per full step, T_{hfs} denotes the holding torque at a full step.

$$\begin{aligned}
T_{inc} &= T_{hfs} \sin\left(\frac{90}{\mu_{pfs}}\right) \\
&= 125 \text{ oz in} \sin\left(\frac{90}{16}\right) \\
&= 12.25 \text{ oz in}
\end{aligned}$$

As such, the selected motors at full $\frac{1}{16}$ microstepping will only be able to provide approximately 10% of the maximum holding torque.

Communication Method Data is required to be transferred between the image processing software and the Arduino controlling the motor assemblies.

Serial The Arduino platform provides a serial interface that allows a read from a USB to serial cable connected directly to a computer. In the case of using a single computational device, this solution is very simple and reliable. However, as this only works well between two devices, expanding the system to communicate with other devices presents a problem.

I2C The Arduino platform also provides native I2C bus for devices. While this setup is more complex than a simple serial connection, it can allow communication between numerous devices.

Decision Both of the above mentioned communication methods run at suitably large speeds. The expected data size is approximately 32 bits, transmitted 125 times per second, much below the bandwidth of the serial connection at 115.2 kbps. As the expected configuration of data transfer through the system is linear in nature, it is not required that more devices are simultaneously supported. As such, the serial connection method provides the most simple and reliable method of communication between the Arduino and processing unit software.

3.5 Tracking Algorithm Design

3.5.1 Final Design

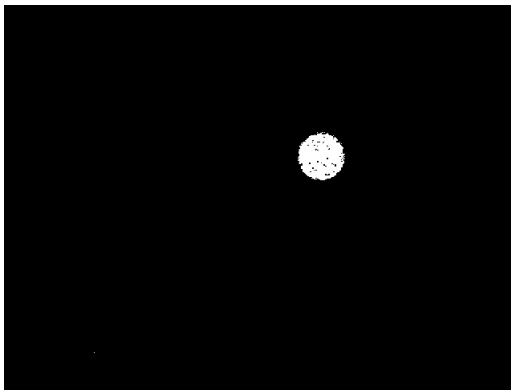
The image processing algorithm is being run on a personal computer. This provides sufficient processing power while allowing for the use of existing hardware. Using a PC also allows for work to be focused on other aspects of the project. In addition, development can be quickly completed and deployed in a familiar environment, saving time when compared to running the algorithm on a dedicated microprocessor or ASIC.

The final design of the object recognition consists of using OpenCV to identify objects using color. Each frame is captured as a matrix which can then be utilized by the program. The image's colors are first converted from RGB (red, green, blue) to HSV (hue, saturation, value) color spaces to allow for faster processing during the puck detection. After this conversion, the image is compared against a hardcoded range of HSV values designed to filter out all colors besides the puck (black). A new matrix is then created which contains only the elements which fell within the specified range. This image is then eroded and dilated, a process which fills any holes caused by imperfections during the masking and refines the edges of the mask. This results in an image which, ideally, contains only the puck and allows for easy tracking and prediction. Figure 7 provides an visual example of this process.

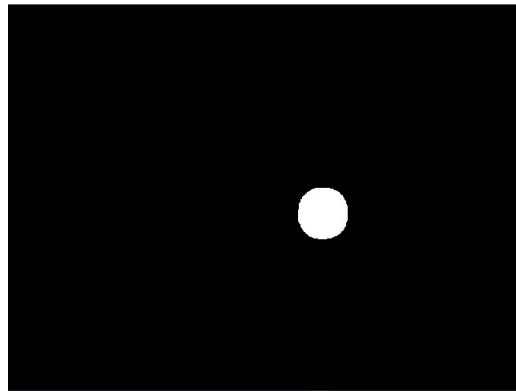
Figure 7: Image Identification Process



(a) Original Image



(b) Image Mask After HSV Filter



(c) Image Mask After Erosion and Dilation

With the mask complete, OpenCV is able to detect the image's contours and calculate the first order moments of the mask. Using this information the centroid of the puck can be calculated as an x,y coordinate which can then be used to predict the future puck location. Figure 8 demonstrates how the image's moments can be used to find the location and centroid of the object.

Detecting the table was done using process similar to the puck's detection. By attaching distinctly colored markers to the edges of the table another HSV filter is used to separate these edge identifiers from the rest of the image and a mask containing only the edges is

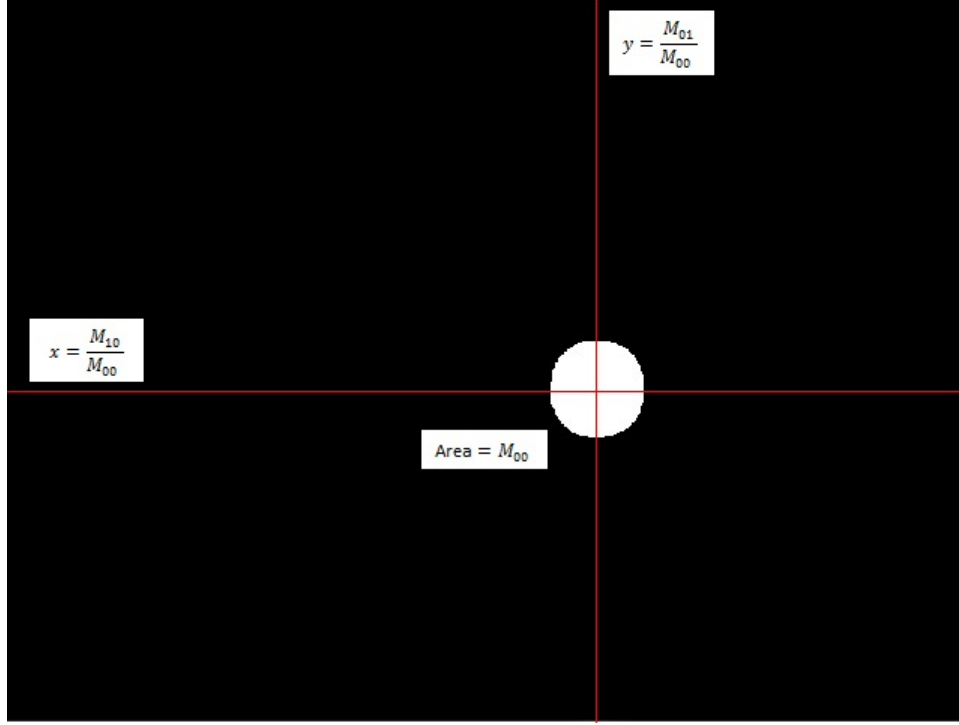


Figure 8: Sample Mask With Area and Coordinate Calculations

created. Using OpenCV, the centroid of each marker is calculated as an (x, y) coordinate and then the edges of the table are constructed by using the four marker centroids to construct a shape. This boundary can then be used in the game state algorithm to determine puck trajectory.

The table detection is performed each time the program is launched in order to ensure the edge coordinates for the table are correct for each execution. This process is performed prior to the execution of the program's central while loop in which puck detection is not active and only the edge markers are being masked. This allows for easy verification of the edge locations before the primary tracking algorithm begins, ensures there will be no false-positives, and allows the edge detection to be performed without an upper limit on time or algorithm performance.

3.5.2 Engineering Design Process

The image tracking portion of the project contained numerous design decisions as there is no well-defined software stack. There are numerous possible ways to implement high-level object recognition and lower-level tracking algorithms.

After selecting the PlayStation Eye as our camera, the first major design decision involved selecting the best object recognition method. Distinguishing the puck from the table and mechanics is essential in order to predict the puck's position and have the paddle respond accordingly. Most importantly, the tracking method has to recognize the puck even when

the image becomes distorted due to high puck speeds.

Multiple object recognition methods exist, but the three considered were motion tracking, shape recognition, and color recognition. Motion tracking, which functions by identifying moving objects relative to a stationary background, was quickly discarded. Initial research indicated there would be no simple way to track the motion of the puck without interference from the moving actuators and paddle. An additional recognition method would have to be used to isolate the puck, making the motion tracking unnecessary. Frames of the puck in motion were used to determine whether shape or color recognition would be most effective. Despite the camera's high frame rate, the shape of the puck became distorted and unrecognizable at high speeds. However, the color was still easily identifiable, making color recognition the ideal choice. Black was chosen as the puck's color, as it would be the only black object in the play area, making it easy to identify.

With color recognition selected, the next major design decision was how to implement the recognition software. After investigating computer color vision, it was apparent that coding robust color recognition software used deep magic [13] that required extensive image processing knowledge outside the scope of this project component. A set of libraries called OpenCV [14] was determined to be the most appropriate solution, as it provided a well-documented API for image processing in C++. These libraries provide algorithms for transforming images as well as simple shape/edge detection. OpenCV is also entirely open source, meaning modifications can be made to the image processing algorithms if required. As OpenCV is only a set of libraries it still requires the object recognition and tracking software to be designed and coded in its entirety.

Although identifying the puck is the primary purpose of the tracking algorithm, it is also important to locate the edges of the table so the prediction algorithm is able to deal with puck-table collisions. The primary decision was whether to use static or dynamic detection for the edges. With static detection, the edges locations are initialized at the beginning of the program and do not change during execution. This is opposed to dynamic detection, in which the positions are continually checked and compared against the position of the puck.

Static detection was chosen for the solution, as the camera mount is designed to hold the camera in a fixed position relative to the table's surface, meaning the edge positions should not change during execution. Dynamic detection would waste CPU cycles re-computing coordinates which should remain constant throughout the program's operation.

Using static detection, a method for initializing the table's coordinates in the program had to be selected. Although the mount is designed to stay in a set position on the table, there is a chance that the camera will shift between uses. Because of this, hardcoding the coordinates was not a feasible solution, implying that the table's position had to be detected on startup. There are a variety of ways to accomplish this, two of which were considered: marking the edges with a distinct color or detecting the white surface of the play area. Due to inconsistent lighting and the surface not being uniformly white, it was difficult to get a shape with well-defined edges when trying to detect the play area. However, by masking only the edges, it was easy to use OpenCV's color and shape detection to get an accurate representation of the table's boundaries.

3.6 Game State Algorithm Design

The game state algorithm takes the position of the puck as input from the image processing algorithm, keeps an internal state of the game, and outputs a reference position of the puck to the paddle control unit.

3.6.1 Final Design

In the final design of the game state algorithm, the game algorithm itself will be written on the PC, running within the program that the image processing algorithm is hosted on. The program will send the data through a serial port to the paddle control unit each time the image processing algorithm completes running.

The specifications for the game state design require only a defensive mode of operation, in which the algorithm calculates the expected trajectory of the puck, and moves the paddle to intercept the puck, moving along the goal line.

An additional, non-essential specification for the algorithm requires an offensive mode of operation. In this mode, when an incoming puck is detected, the algorithm will select an angle to return the puck at based off of its initial position and velocity. When selecting an angle, the position of the opponent's paddle is approximated from the initial trajectory of the puck, and the selected angle will generally avoid this area. The angles of the return trajectory are calculated using a varying number of bounces off of each side.

From the initial position and velocity of the puck, an approximate range of positions for the player's puck at the time of striking the puck can be determined. Given a desired number of bounces and the length of the table, the desired angle of attack for such a path can be determined by the following equation, where θ represents the desired angle of puck velocity taken from the line perpendicular to the goal, L represents the width of the table, the tuple (x_p, y_p) represents the position of the puck on the table, with x being the position with respect to the width of the table from the left side, y being the position and (x_g, y_g) represents the position of the goal, and n represents the number of bounces of the table on the right hand side.

$$\theta = \tan^{-1}\left(\frac{y_g - y_p}{x_p - x_g + nL}\right)$$

By calculating a number of potential valid θ values for different values of $n \in [-2, 2]$ where n is an integer, determining which ones are feasible from the current positions of both the paddle and puck, and ideally selecting a feasible angle nondeterministically. Once the value of θ is calculated, the algorithm should position the paddle at a point along the trajectory of the puck such that it will impact the puck so that the normal vector perpendicular to the tangent of the surface of impact is equidistant from both the trajectory of the puck and the desired trajectory.

In both modes of operation, the velocity of the puck can be determined from the difference between positions received from the image processing algorithm, and the amount of time

passed between frame captures. The camera being used should return frames at a relatively constant rate, so the amount of time can be determined by the number of frames between the last position state and the currently processed state, and the frame rate the camera is running at. Based off of the velocity of the puck and the current position of the puck, the algorithm will calculate a position that the paddle can attempt to move to that is on the predicted line of motion of the puck.

3.6.2 Engineering Design Process

With regards to the design of the game state algorithm, the primary decision involved the logical and physical flow of data of the system, determining the computational platform the game state algorithm and motor control algorithm should run on. Regardless of the design of the game state algorithm, the expected functional behaviour is clear. The game state algorithm should take the position of the puck as input, and output a desired puck position to the motor controller. With regards to the logical layout of the data flow in the algorithm, the decision was reduced to three potential options. The game state algorithm could either exist on the PC and send its data to a motor control algorithm also running on a PC, the algorithm could exist on the PC and send its data to a motor control algorithm running on a microcontroller, or both the game algorithm and the motor control algorithm could function on the microcontroller. In order to weigh the pros and cons of each option, a decision matrix was created. The weighting of the decision matrix is as follows:

Table 4: Game State Criteria Weight

Category	Weight
Simplicity of Implementation	25
System Performance	45
Robustness	20
Resources Needed	10

One Platform Solution Implementing both the image processing and game state algorithm on the same platform makes communication very simple, in comparison with the other solutions. The one function performing the color tracking detection algorithm can pass coordinates giving the location of the puck to the game state algorithm, which has an internal state machine tracking the current state of the game. Depending on the platform that the image processing algorithm is running on, the performance of the machine may decrease as a result of adding the state machine, depending on the hardware limitations of the platform and the complexity of the algorithm being run. For example, if the design was to switch onto an FPGA, the extra complexity of the game state machine would potentially require rerouting the hardware on the chip to accommodate the new logic for the game state machine. However, as the platform being used is a PC, the hardware will likely be fast enough

to handle the additional stress of the game machine, considering that the state machine is driven by the input, and so should only change once per update. Despite this, the image processing platform must be able to both reliably produce the position of the puck from a 320 x 240 px or larger image.

The one platform solution completely eliminates the need for a microcontroller, having the target platform interfacing directly with the motor controller. This does require an implementation of a motor driver, however, which requires additional effort, reducing the simplicity of the overall design. As this implementation requires a substantial amount of effort and code to be written, the solution gets a score of 5 for simplicity, being significantly more complex than alternatives. As all of the tasks are operating on the same machine, communication between algorithmic processing blocks is simple and quick. However, as the performance of the algorithm is subject to the scheduling of the operating system running on the PC, there may be potential performance issues. As such, this solution gets a score of 40 in performance. In terms of robustness, this solution is the most robust, as data is extremely unlikely to be lost at any point outside of high latency for dropped frames. This solution receives 10 points for the robustness of the design. As this solution only relies on one platform, the solution gets a score of 10 for the lack of additional resources required.

Slave Microcontroller Solution Having the game state algorithm on the PC gives the benefits mentioned above, but keeping the motor control unit on the microcontroller adds the requirement of communications between the image processing platform and the microcontroller. As the communications can be handled through a serial connection, which is relatively easy to implement, the increase in complexity is not as substantial as writing a motor driver for the PC-only solution. This solution receives a 20 in simplicity, due to aforementioned reasons in addition to the simple programming interface offered for the microcontroller. This system is likely the most well-performing solution, as the limited hardware of the microcontroller is being used only to drive the motor, and the more robust PC hardware is being used for the larger image processing and game state algorithm work. This results in a performance score of 45. The robustness of the system is limited by the serial communications used to communicate position data between the PC and the microcontroller. The additional communication link introduces opportunities for missed or dropped frames, so the solution gets a robustness score of 5. Due to the inclusion of a microcontroller, this solution gets a score of 0 for additional resources required.

Master Microcontroller Solution Implementing the game state algorithm on a microcontroller frees the image processing platform from performing additional tasks. As the two entities are implemented on separate platforms, though, communication may be an issue. Due to the relatively slow speed of the game state, with a frame from the camera incoming at a potential maximum frequency of 120 Hz, the software running on the microcontroller should be able to handle the incoming data with relative ease. Despite this, the hardware on the microcontroller may not handle the floating point operations required for the trigonometric calculations necessary to find angles of attack. This reduces the performance to a value of 30, as the microcontroller is likely capable of the task, but must be tested to confirm func-

tionality. The implementation of both the game state and motor control algorithm on the microcontroller results in simple implementations due to existing libraries for motor control implementation and simple programming interfaces. This gives the master microcontroller solution a 25 for simplicity. The robustness of this solution, as in the slave microcontroller solution, is limited due to the use of inter-device communication. From this, the solution gets a score of 5 in robustness. As additional resources from the microcontroller are being used, the score for additional resources is reduced to 0.

Logical Flow Scoring Given the considerations stated above, the final weighting of the options can be summarized in Table 5.

Table 5: Final Game State Option Weights

Implementation	Simplicity	Performance	Robustness	Resources	Total
One Platform	5	40	10	10	65
Slave Microcontroller	20	45	5	0	70
Master Microcontroller	25	30	5	0	60
Possible Weighting	25	45	20	10	100

As it had the highest score, the slave microcontroller solution was selected for the logical flow of the system.

4 Discussion and Project Timeline

4.1 Discussion on Design

4.1.1 Hours Worked

Table 6 below summarizes the amount of hours worked as of July 3rd, 2014.

Table 6: Summary of Hours Worked

Group Member	Hours Worked
--------------	--------------

(names removed)

4.1.2 Critical Design Evaluation

As it stands, the design should satisfy the majority of the previously listed essential specifications. Most of the specifications involving the camera hardware, motor assembly, and rail system have been met by researching and selecting hardware which is most appropriate for the situation. Likewise, the specifications relating to the camera mount are met by the mount's design, and there should be no risk of failing to meet them during implementation.

Specifications involving the processing unit have all been demonstrated in initial testing, and should experience no change during the transition to the final design. Although less specific, all but two of the essential image processing specifications have been met in practice. Currently, it is unknown whether the final tracking algorithm will consistently be able to predict puck position with a maximum error of $\pm 25\%$ of the puck's diameter. The accuracy of the tracking may degrade when the puck is moving at high speeds, but it is expected this specification will be met once the performance of the final design is verified. Similarly, the project is not far enough along yet to fully verify the functionality of the game state algorithm. However, this is arguably the most crucial aspect of the project and significant effort will be made to ensure this is met in the unlikely case our current design does not fulfill this requirement. Specifics relating to the non-essential "aggressive play mode" specification are also unverified meaning the design may require modification after the initial implementation. This is expected as the aggressive mode will undoubtedly require fine-tuning and modification once the algorithm's behavior can be observed in practice.

The only other specification which has yet to be demonstrated is the motor assembly's speed. It is expected that the 5 m/s paddle velocity will be provided by the motor but it is possible this speed will be unreachable. In this case the project should still function, but it may not be able to block all possible shots. It may also be possible to offset this slower speed by making the tracking algorithm more robust. In the worst case scenario, a motor with higher performance could be purchased in order to meet this specification.

While this project is not novel due to existing similar solutions, there are certain aspects of the design that are creative. For example, the use of the camera system to provide feedback for the paddle control unit is an elegant solution that does not require the use of specialized hardware such as a rotary encoder or potentiometer. Another creative aspect of the proposed solution is the potential implementation of a nondeterministic, "smart" algorithm for offensive gameplay. This adds an element of "randomness" to the game that prevents the behaviour of the robot from being predictable and avoids monotonous gameplay. This helps in the emulation of a skilled human opponent. The solution is elegant as the design consists of many modular and abstracted components which are constructed entirely out of commercially available hardware. This modularity allows for easy development and troubleshooting, in addition to a clean and easy to follow design.

This project covers many elements of electrical and computer engineering and is undeniably an open-ended design problem as there are many ways to approach the problem and develop a solution. Upper-year engineering knowledge was used in the design of the linear actuator, as DC motors are part of the ECE 463 syllabus. Image processing in general is a wide field, and the implementation of color detection and puck position tracking can relate to concepts from ECE 417 and ECE 413. Interfacing with embedded components and dealing with synchronization is covered in ECE 455 and 224/324. In addition, the design of the game state and image processing algorithms will likely involve concepts from ECE 406, introduced in ECE 250.

4.1.3 Potential Hazards

There are several potential hazards inherent to a project such as this. Firstly, the moving parts that control the paddle are a hazard as they could potentially injure someone who puts their hands in the area during operation. This hazard can be mitigated by only allowing people near the human player end of the table while the system is in operation. Another issue that may arise is that the puck could fly off the table if hit too hard. Anyone who tests the system will be instructed not to hit the puck at full strength while playing, and the robot will be designed such that it cannot return the puck with enough force to launch it. The last hazard is that the majority of the components are powered using standard 120 V outlets. This is necessary because the air hockey table itself must be powered via an outlet (this is existing circuitry on the table), and the computing device used for the tracking software will also require power via an outlet (again using existing circuitry). The motor drivers also require a certain amount of voltage, which will be supplied to them via CSA/ESA approved power supplies. In order to allow for this many power connections, a surge protected power bar will be used to supply power to the system.

4.2 Project Timeline

Figure 9 and Figure 10 are Gantt charts that describe the relationships between the project stages and their timelines for 4A and 4B respectively. Any work done over the fall co-op term is likely to be minor code development and optimizations that were deemed unnecessary to include in the Gantt charts. However, such work could result in tweaks to the 4B timeline.

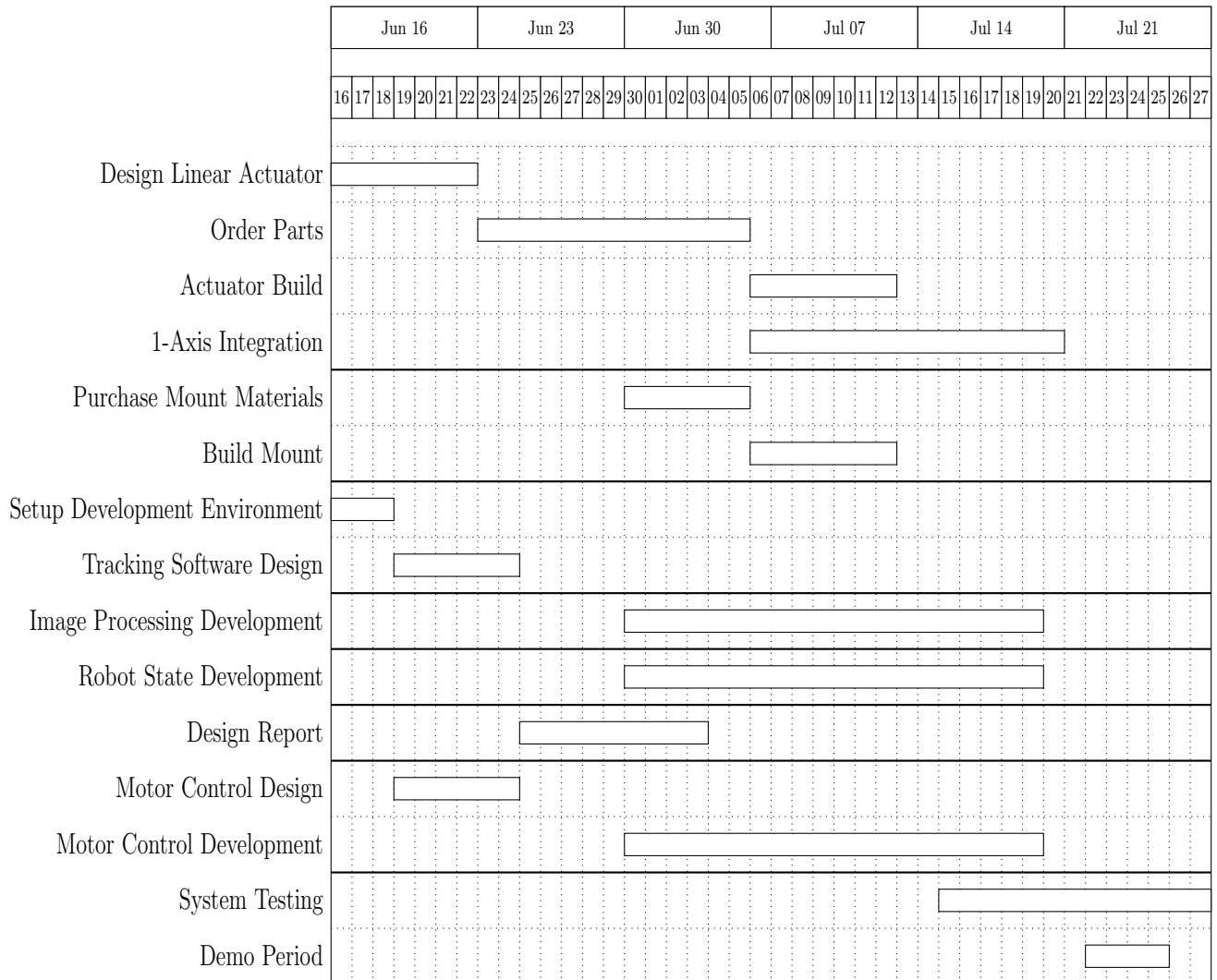


Figure 9: 4A Project Gantt Chart (June 16 - July 30)

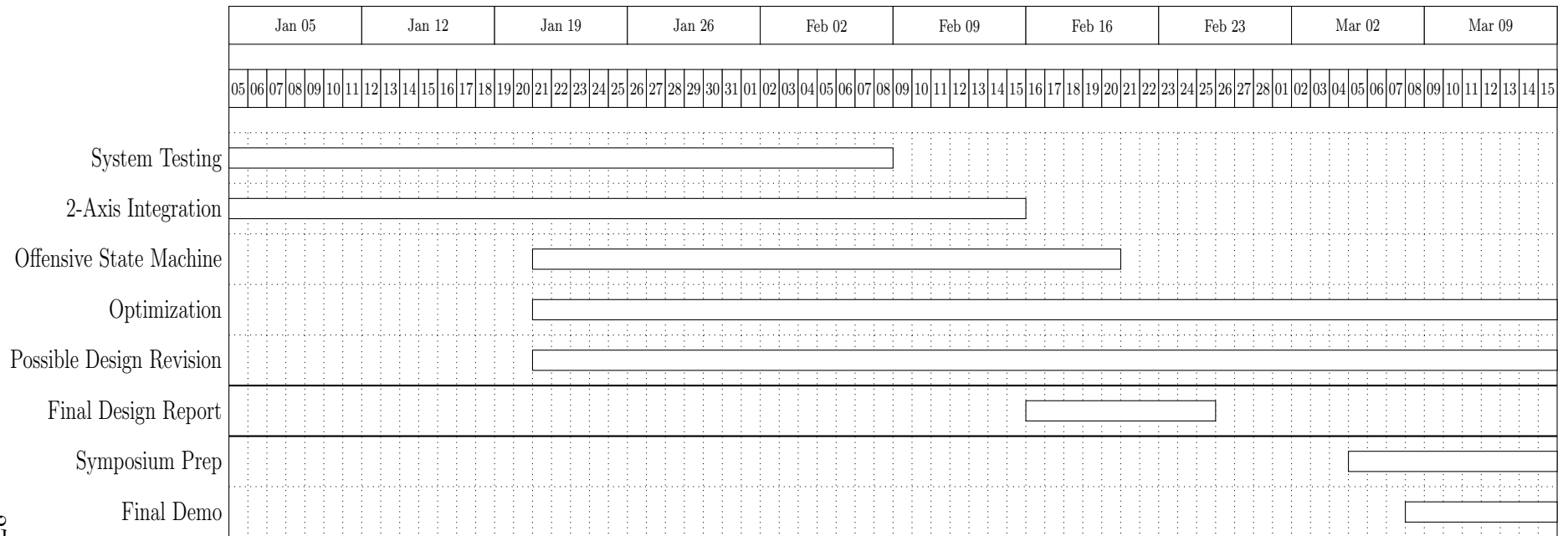


Figure 10: 4B Project Gantt Chart (Jan 05 - March 08)

References

- [1] *Frequently Asked Questions* [Online]. Available: <http://www.airhockeyworld.com/ahfaq.asp#2>.
- [2] J. Falconer. (2013, Jun 13). *This Robot Wants to Beat You at Air Hockey* [Online]. Available: <http://spectrum.ieee.org/automaton/robotics/artificial-intelligence/japanese-air-hockey-robot>.
- [3] J. Julio. (2014, Feb 3). *New Project: Air Hockey Robot (a 3D printer hack)* [Online]. Available: <http://cienciaycacharreo.blogspot.com.es/2014/02/new-project-air-hockey-robot-3d-printer.html>.
- [4] *12V 0.68A 125oz-in Unipolar Stepper Motor* [Online]. Available: <http://www.robotshop.com/ca/en/rbsoy14-soyo-unipolar-stepper-motor.html>.
- [5] *Open Builds Part Store* [Online]. Available: <http://openbuildspartstore.com>.
- [6] *V-Slot CAD Models for Autodesk Inventor* [Online]. Available: <http://www.openbuilds.com/resources/v-slot-cad-models-for-autodesk-inventor.4/>.
- [7] *Big Easy Driver - ROB-11876* [Online]. Available: <https://www.sparkfun.com/products/11876>.
- [8] *Playstation Eye* [Online]. Available: https://en.wikipedia.org/wiki/PlayStation_Eye.
- [9] *AccelStepper library for Arduino* [Online]. Available: <http://www.airspayce.com/mikem/arduino/AccelStepper/>.
- [10] *Generate stepper-motor speed profiles in real time* [Online]. Available: http://fab.cba.mit.edu/classes/MIT/961.09/projects/i0/Stepper_Motor_Speed_Profile.pdf.
- [11] *How to prevent step losses with Stepper Motors?* [Online]. Available: http://static.micromo.com/media/wysiwyg/Technical-library/Stepper/Stepper_Why%20Steppers%20Loose%20Steps_WP.pdf.
- [12] *Microstepping: Myths and Realities* [Online]. Available: <http://www.micromo.com/microstepping-myths-and-realities>.
- [13] *Deep magic* [Online]. Available: http://en.wikipedia.org/wiki/Deep_magic.
- [14] *OpenCV* [Online]. Available: <http://opencv.org/>.